운영체제

Assignment 2

담당교수님 : 최상호 교수님

신윤석

2019202053

컴퓨터정보공학부

내용

운영체제 1

Assignment 2 1

Introdution 3

Implementation 3

Result 6

고찰 9

Reference 10

Introdution

이번 과제에서는 모듈을 만들어 커널에 적재시킨다. 모듈은 ftrace라는 시스템 콜을 wrapping하는 모듈이다. 이를 위해서는 커널에 ftrace라는 시스템 콜을 우선 만든 뒤 컴파일 해서 컴퓨터에 컴파일 된 커널을 설치하는 것이 선행되어야 한다. 또한, 과제로 만들어야 하는 파일을 두개를 제시했기 때문에 나오는 모듈도 두개이며, 각 모듈간 통신도 가능해야 한다. 이를 위해 EXPORT_SYMBOL을 이용하여 다른 모듈에서도 읽어올 수 있는 변수를 만들어야 한다.

결론적으로 ftrace라는 system call을 통해 구현하고 싶은 것은 지목된 프로세스가 하는 system call을 추적하는 것이다. open, read, write, lseek, close 시스템 콜을 몇 번 사용했는지, 무슨 파일을 open했는지, 얼마나 읽고 썼는지 등을 커널 메시지로 출력하면 된다.

Implementation

이번 과제에서 해야 할 일은 크게 두가지로 구분된다. 커널에 336번 system call인 ftrace를 추가하는 것과 추가된 ftrace systemcall을 hooking하는 module을 만드는 것이다. 우선 system call을 만드는 작업부터 시작한다.

우선 /arch/x86/entry/syscalls/syscall_64.tbl에 접근해서 336번 위치에 새로운 system call인 ftrace를 등록해준다.

```
os2019202053@ubuntu: ~/linux-4.19.67
322     64      execveat                __x64_sys_execveat/ptregs
323     common  userfaultfd             __x64_sys_userfaultfd
324     common  membarrier              __x64_sys_membarrier
325     common  mlock2                  __x64_sys_mlock2
326     common  copy_file_range         __x64_sys_copy_file_range
327     64      preadv2                 __x64_sys_preadv2
328     64      pwritev2                __x64_sys_pwritev2
329     common  pkey_mprotect           __x64_sys_pkey_mprotect
330     common  pkey_alloc              __x64_sys_pkey_alloc
331     common  pkey_free               __x64_sys_pkey_free
332     common  statx                   __x64_sys_statx
333     common  io_pgetevents           __x64_sys_io_pgetevents
334     common  rseq                    __x64_sys_rseq
336     common  ftrace                  __x64_sys_ftrace

#
# x32-specific system call numbers start at 512 to avoid cache impact
# for native 64-bit operation. The __x32_compat_sys stubs are created
# on-the-fly for compat_sys_*() compatibility system calls if X86_X32
# is defined.
#
512     x32     rt_sigaction            __x32_compat_sys_rt_sigaction
513     x32     rt_sigreturn            sys32_x32_rt_sigreturn
                                                        333,1         90%
```

다음은 /include/linux/syscalls.h 파일에 system call을 받았을 때 실행할 함수선언을 해준다.

```
os2019202053@ubuntu: ~/linux-4.19.67
        if (force_o_largefile())
                flags |= O_LARGEFILE;
        return do_sys_open(AT_FDCWD, filename, flags, mode);
}

extern long do_sys_truncate(const char __user *pathname, loff_t length);

static inline long ksys_truncate(const char __user *pathname, loff_t length)
{
        return do_sys_truncate(pathname, length);
}

static inline unsigned int ksys_personality(unsigned int personality)
{
        unsigned int old = current->personality;

        if (personality != 0xffffffff)
                set_personality(personality);

        return old;
}
asmlinkage int ftrace(pid_t pid);
#endif
                                                        1296,1        바닥
```

다음은 임의의 위치에 ftrace함수의 원형을 선언해준다.

```
os2019202053@ubuntu: ~/linux-4.19.67
#include <linux/kernel.h>
#include <linux/syscalls.h>

// origianl ftrace function call
SYSCALL_DEFINE1(ftrace, pid_t, usrPid)
{
        printk("ftrace function is called! the parameter is %d\n", usrPid);
// it just prints that it's called
        return 0;
}
~
~
~
~
~
~
~
~
~
~
~
~
~
"ftrace/ftrace.c" 9L, 246C
```

그리고 해당 디렉토리에 Makefile을 만들어서 obj-y 항목에 방금 만든 파일 이름.o 를 넣어준다.

```
obj-y := ftrace.o

obj-m := ftracehooking.o iotracehooking.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
	$(MAKE) -C $(KDIR) SUBDIRS=$(PWD) modules

clean:
	$(MAKE) -C $(KDIR) SUBDIRS=$(PWD) clean

CFLAGS: -DEXPRT_SYMTAB

"~/HW02/linux-4.19.67/ftrace/Makefile" 15L, 250C          1,1          모두
```

다음으로는 /Makefile을 열어서 ftrace 함수 원형이 정의된 소스파일과 Makefile이 있는 디렉토리의 주소를 core-y열에 추가해준다.

```
os2019202053@ubuntu: ~/linux-4.19.67
                echo "int main() {}" | $(HOSTCC) -xc -o /dev/null $(HOST_LIBELF_
LIBS) -,1,0)
  ifeq ($(has_libelf),1)
    objtool_target := tools/objtool FORCE
  else
    SKIP_STACK_VALIDATION := 1
    export SKIP_STACK_VALIDATION
  endif
endif


ifeq ($(KBUILD_EXTMOD),)
core-y          += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ block/ ftrace/

vmlinux-dirs    := $(patsubst %/,%,$(filter %/, $(init-y) $(init-m) \
                     $(core-y) $(core-m) $(drivers-y) $(drivers-m) \
                     $(net-y) $(net-m) $(libs-y) $(libs-m) $(virt-y)))

vmlinux-alldirs := $(sort $(vmlinux-dirs) $(patsubst %/,%,$(filter %/, \
                     $(init-) $(core-) $(drivers-) $(net-) $(libs-) $(virt-))))

init-y          := $(patsubst %/, %/built-in.a, $(init-y))
core-y          := $(patsubst %/, %/built-in.a, $(core-y))
                                                          963,71-79      55%
```

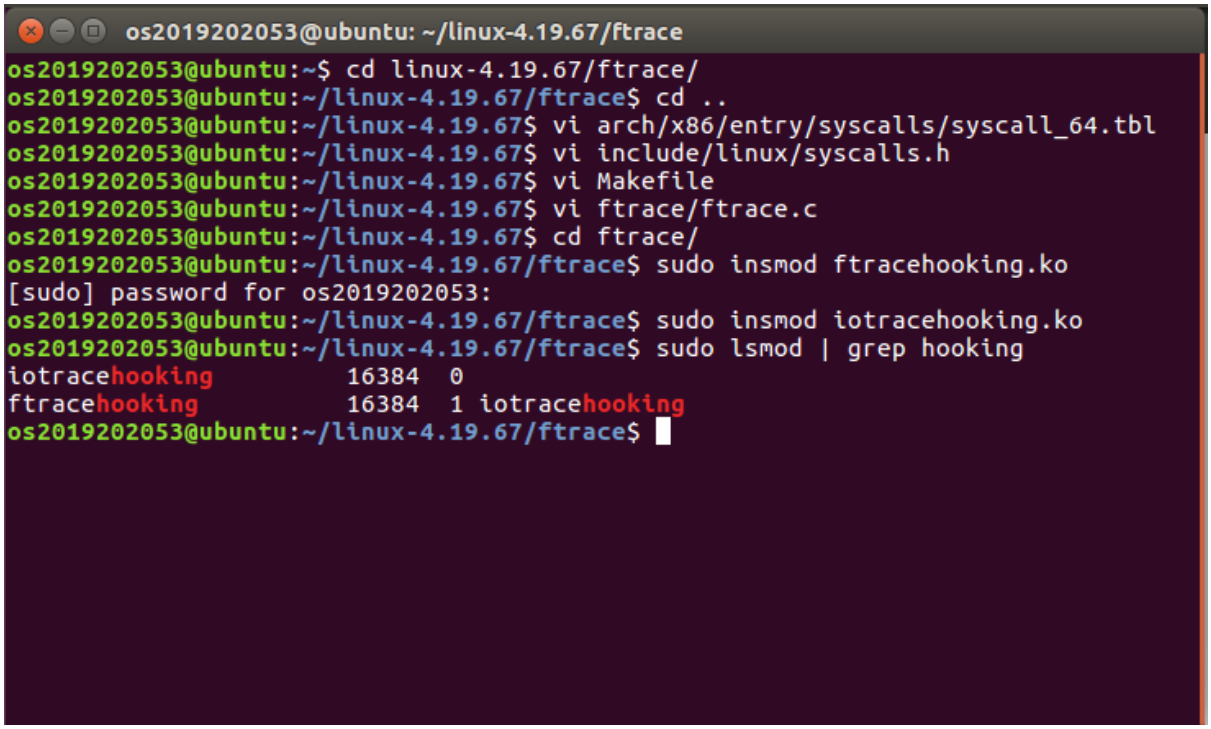
다음으로는 커널 컴파일 해서 커널을 적재하고 컴퓨터를 재부팅 하면 ftrace라는 시스템 콜 추가가 완료된다.

다음으로는 ftrace를 hooking하는 모듈을 만들어서 모듈을 커널에 적재해주면 된다. 이때, 과제 요구사항으로 ftracehooking.h파일을 하나 만들고 ftracehooking.c, iotracehooking.c파일을 각각 만

들라는 사항이 있었다. 이를 만족시키기 위해 헤더에는 ftracehooking.c와 iotracehooking.c 에서 공통적으로 사용되는 헤더와 include와 공통으로 쓰이는 함수 선언 및 정의를 해 주었다. 또한, ftacehooking.c에서는 ftace 함수 원형을 wrapping할 함수를 선언하고 정의하게 된다. 이 함수는 프로그램 전역변수인 tracePID가 0일 때 함수 call로 pid를 받으면 해당 pid를 가진 프로그램을 trace하기 시작한다. 이때 iotracehooking.c에 정의된 함수로 wrapping된 open, close, read, write, lseek함수가 ftracehooking.c에 선언된 변수에 실행 정보를 넘겨주면서 프로세스 추적 기록을 남기게 된다. 다시 system call과 동시에 0을 시스템 콜 함수 인자로 넘겨주면 프로세스 추적이 끝나면서 추적 기록을 커널 메시지로 출력하게 된다.

Result

우선 모듈을 적재하는 모습이다.



새로 만든 모듈이 잘 적재되었다.

이 상태에서 다음과 같은 코드를 가지고 만든 프로그램을 실행시켜 본다.

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/syscall.h>

int main()
{
        syscall(336, getpid());
        int fd = 0;
        char buf[50];
        fd = open("abc.txt", O_RDWR);
        for (int i = 1; i <= 5; ++i)
        {
                read(fd, buf, 5);
                lseek(fd, 0, SEEK_END);
                write(fd, buf, 5);
                lseek(fd, i*5, SEEK_SET);
        }
        lseek(fd, 0, SEEK_END);
        write(fd, "HELLO", 6);
        close(fd);
        syscall(336,0);
        return 0;

}
```

이 코드는 ftracing을 하기 위해 과제와 함께 올라온 코드이다. 잘 보면 open은 “abc.txt”를 열기 위해 한번, read는 5byte씩 5번, write는 5byte씩 5번, 6byte를 한번 해서 총 6번, lseek는 11번, close는 한번 실행하는 것을 확인할 수 있다. 과연 그렇게 나오는지 확인해 보겠다.

```
os2019202053@ubuntu: ~/Downloads
os2019202053@ubuntu:~/Downloads$ gcc test.c -o run
os2019202053@ubuntu:~/Downloads$ ./run
os2019202053@ubuntu:~/Downloads$ dmesg |tail -n4
[ 4089.524021] OS Assignment 2 ftrace [4097] Start
[ 4089.538066] [2019202053] /run file[abc.txt] stats [x] read - 25 / written - 3
1
[ 4089.538067] open[1] close[1] read[5] write[6] lseek[11]
[ 4089.538067] OS Assignment 2 ftrace [4097] End
os2019202053@ubuntu:~/Downloads$
```

ftrace가 잘 되었음을 확인할 수 있다. 다시한번 dmesg 전체 결과를 살펴보면

```
os2019202053@ubuntu: ~/Downloads
[    20.810850] kvm: Nested Virtualization enabled
[    20.810865] kvm: Nested Paging enabled
[    20.864257] Decoding supported only on Scalable MCA processors.
[    20.905377] Decoding supported only on Scalable MCA processors.
[    20.957426] Decoding supported only on Scalable MCA processors.
[    21.013754] Decoding supported only on Scalable MCA processors.
[    21.076713] Decoding supported only on Scalable MCA processors.
[    21.113597] Decoding supported only on Scalable MCA processors.
[    24.484642] Adding 998396k swap on /dev/sda5.  Priority:-2 extents:1 across:9
98396k FS
[    33.772887] IPv6: ADDRCONF(NETDEV_UP): ens33: link is not ready
[    33.777503] IPv6: ADDRCONF(NETDEV_UP): ens33: link is not ready
[    33.780982] e1000: ens33 NIC Link is Up 1000 Mbps Full Duplex, Flow Control:
None
[    33.781834] IPv6: ADDRCONF(NETDEV_CHANGE): ens33: link becomes ready
[  3743.364872] ftracehooking: loading out-of-tree module taints kernel.
[  3743.364914] ftracehooking: module verification failed: signature and/or requi
red key missing - tainting kernel
[  4089.524021] OS Assignment 2 ftrace [4097] Start
[  4089.538066] [2019202053] /run file[abc.txt] stats [x] read - 25 / written - 3
1
[  4089.538067] open[1] close[1] read[5] write[6] lseek[11]
[  4089.538067] OS Assignment 2 ftrace [4097] End
os2019202053@ubuntu:~/Downloads$
```

커널 메시지가 잘 쓰여져 있다.

마지막으로 모듈 제거도 잘 되는 모습을 확인할 수 있다.

```
os2019202053@ubuntu: ~/linux-4.19.67/ftrace
os2019202053@ubuntu:~$ cd linux-4.19.67/ftrace/
os2019202053@ubuntu:~/linux-4.19.67/ftrace$ cd ..
os2019202053@ubuntu:~/linux-4.19.67$ vi arch/x86/entry/syscalls/syscall_64.tbl
os2019202053@ubuntu:~/linux-4.19.67$ vi include/linux/syscalls.h
os2019202053@ubuntu:~/linux-4.19.67$ vi Makefile
os2019202053@ubuntu:~/linux-4.19.67$ vi ftrace/ftrace.c
os2019202053@ubuntu:~/linux-4.19.67$ cd ftrace/
os2019202053@ubuntu:~/linux-4.19.67/ftrace$ sudo insmod ftracehooking.ko
[sudo] password for os2019202053:
os2019202053@ubuntu:~/linux-4.19.67/ftrace$ sudo insmod iotracehooking.ko
os2019202053@ubuntu:~/linux-4.19.67/ftrace$ sudo lsmod | grep hooking
iotracehooking         16384  0
ftracehooking          16384  1 iotracehooking
os2019202053@ubuntu:~/linux-4.19.67/ftrace$ sudo rmmod iotracehooking
os2019202053@ubuntu:~/linux-4.19.67/ftrace$ sudo rmmod ftracehooking
os2019202053@ubuntu:~/linux-4.19.67/ftrace$ sudo lsmod |grep hooking
os2019202053@ubuntu:~/linux-4.19.67/ftrace$
```

고찰

이번과제는 강의자료만 보고 완성하기에는 어려움이 있는 과제였다. 가장 큰 어려움은 system call함수의 argument전달 방법이었다. 실습에서는 SYSCALL_DEFINE1이나 SYSCALL_DEFINEx 등의 메크로로 시스템 콜 함수를 선언한다든가 /include/linux/syscalls.h 파일에 cscope로 open함수등의 system call 함수 선언을 보면

```
/* __ARCH_WANT_SYSCALL_NO_AT */
asmlinkage long sys_open(const char __user *filename,
                         int flags, umode_t mode);
```

와 같이 나와있는데 정작 과제 요구사항으로는

*static asmlinkage long ftrace_open(const struct pt_regs *regs)*

와 같은 함수선언을 사용하라고 해서 한참 구글링 했다. 리눅스 커널 4.17.몇 까지는 저렇게 parameter로 값을 넘기는 방식으로 작동했는데 4.17.몇 이후부터는 그렇게 하면 안되고 struct pt_regs 포인터로 넘겨줘야 한다고 한다. 그래서 그런지 이 방법을 알기 전까지는 함수 parameter에 접근하려고만 하면 모듈을 적재하는 순간 시스템이 멈춰버렸다. 그런데 그러면 도대체 어디 레지스터에 몇 번째 파라미터가 저장되어 있는지도 문제이다. ARM이었으면 아무생각 없이 r0부터 순서대로 뽑아 썼을 텐데 x86이라 이것도 검색해봤다. 검색해보니까 di, si, dx순서대로 쓴다고 한다.[i] 그래서 해당 register를 참조해 보니까 원하는 정보가 있어서 이 부분도 해결이 되었다. 이제 iotracehooking.c에서도 wrapping함수로 조금 쓸만한 정보를 얻을 수 있었다. 하지만 마지막 난관이 있었다. 바로 어떤 프로세스가 시스템 콜을 했냐를 아는 방법이었다. 이건 검색하기도 뭐 해서 chat gpt에 혹시나 해서 검색해 봤다. 조금 다른 걸 알려주고 있긴 했으나, 결론적으로 current라는 키워드를 쓰면 된다는 사실을 알았다. Current를 어디선가 자주 본거 같아서 강의자료를 확인

해 봤더니 강의 자료에도 조용히 조심스럽게 부모 프로세스의 pid를 얻는 방법 부분에 나와있는 것을 확인했다. 아무튼 이를 이용해서 안심하고 current -> pid == tracePID 인지 검사해서 tracePID와 일치하는 process에서 call 한 system call에 대해서만 추적하는 것이 가능해졌다.

여담으로 검색해보면서 백신프로그램은 system call hooking을 하지 않으면 사실상 구현하기가 힘들다는 이야기를 보았다. 생각해보면 프로그램, 프로세스를 감시하려면 어차피 보안상 위험한 일은 커널이 알아서 다 해주니, 커널을 조금만 손보면 프로세스 감시가 가능해서 그런가 보다 싶다. 그런데 파일 열고 닫기라든가 읽고 쓰기 등은 자주 하는 시스템 콜인 것 같은데 그런 시스템 콜을 조금 긴 함수로 wrapping을 해 버리면 시스템 전체가 느려지질 것 같다는 생각을 하게 되었다. 따라서 최대한 단순하게 wrapping함수를 코딩해야 할 것이다.

Reference

---

[i] x86 호출 규약 - 위키백과, 우리 모두의 백과사전 (wikipedia.org), https://ko.wikipedia.org/wiki/X86_%ED%98%B8%EC%B6%9C_%EA%B7%9C%EC%95%BD, 2023

-> https://web.archive.org/web/20110716085220/http://www.x86-64.org/documentation/abi.pdf

(위키피디아 같은 곳을 reference로 담기엔 신빙성이 떨어진다는 것은 알지만 그럼에도 불구하고 구글링을 하면 제일 먼저 표로 정리되어 떠서 가볍게 참조 걸어놨습니다)